Zellic

Prepared for
Zach Petersen
Raajeev Reddy
Paxos

Prepared by
Filipe Alves
Chongyu Lv
Zellic

March 2, 2026

OFTWrapperUpgradeable
Smart Contract Security Assessment

Contents

About Zellic **4**

1. Overview **4**

1.1. Executive Summary 5

1.2. Goals of the Assessment 5

1.3. Non-goals and Limitations 5

1.4. Results 5

2. Introduction **6**

2.1. About OFTWrapperUpgradeable 7

2.2. Methodology 7

2.3. Scope 9

2.4. Project Overview 9

2.5. Project Timeline 10

3. Detailed Findings **10**

3.1. Test suite lacks integration tests against USDG token 11

3.2. No admin function to reset rate-limit capacity 13

3.3. Token-type declaration does not reflect multitoken deployment 15

3.4. Uninitialized rate limits block cross-chain transfers until explicitly configured 16

3.5. Unchecked return values from Paxos token mint and burn calls 17

4. Threat Model **17**

4.1. Module: OFTWrapperUpgradeable 18

5. Assessment Results **23**

5.1. Disclaimer 24

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the #1 CTF (competitive hacking) team ↗ worldwide in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io ↗ and follow @zellic_io ↗ on Twitter. If you are interested in partnering with Zellic, contact us at hello@zellic.io ↗.

1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Paxos on February 27th, 2026. During this engagement, Zellic reviewed OFTWrapperUpgradeable's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Could an incorrect rate-limit configuration or logic error allow cross-chain transfers to bypass inbound or outbound rate limits?
- Is the UUPS upgradability pattern implemented correctly, preserving state and access control across upgrades?
- Could an attacker cause a lockup of user funds during the send or receive flow?

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- The correctness or security of the LayerZero protocol, endpoint contracts, or messaging infrastructure
- The underlying Paxos token contracts (PaxosTokenV2, PaxosTokenClaimableRewards), which were assumed to be safe for the purposes of this audit.
- Code outside the diff between commits `64d8dc5` and `2c919aa`
- Front-end components
- Infrastructure relating to the project
- Key custody

Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

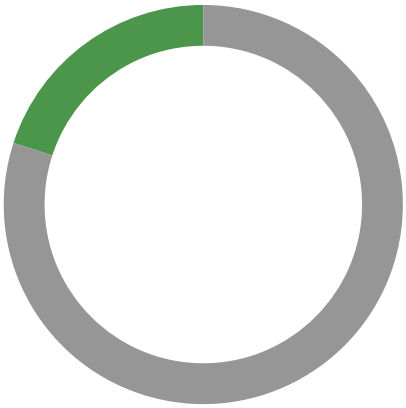
1.4. Results

During our assessment on the scoped OFTWrapperUpgradeable contracts, we discovered five findings. No critical issues were found. One finding was of low impact and the other findings were

informational in nature.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	0
Low	1
Informational	4



2. Introduction

2.1. About OFTWrapperUpgradeable

Paxos contributed the following description of OFTWrapperUpgradeable:

> OFTWrapperUpgradeable serves as a proxy for LayerZero's OFT standard, allowing LayerZero to mint and burn tokens as usual, which includes the same AML standards as a same chain transfer. When a LayerZero mint or burn request is initiated the OFTWrapperUpgradeable forwards it to the underlying token contract. For this to work, the underlying token must authorize OFTWrapperUpgradeable to perform minting and burning operations on its behalf.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Basic coding mistakes. Many critical vulnerabilities in the past have been caused by simple, surface-level mistakes that could have easily been caught ahead of time by code review. Depending on the engagement, we may also employ sophisticated analyzers such as model checkers, theorem provers, fuzzers, and so on as necessary. We also perform a cursory review of the code to familiarize ourselves with the contracts.

Business logic errors. Business logic is the heart of any smart contract application. We examine the specifications and designs for inconsistencies, flaws, and weaknesses that create opportunities for abuse. For example, these include problems like unrealistic tokenomics or dangerous arbitrage opportunities. To the best of our abilities, time permitting, we also review the contract logic to ensure that the code implements the expected functionality as specified in the platform's design documents.

Integration risks. Several well-known exploits have not been the result of any bug within the contract itself; rather, they are an unintended consequence of the contract's interaction with the broader DeFi ecosystem. Time permitting, we review external interactions and summarize the associated risks: for example, flash loan attacks, oracle price manipulation, MEV/sandwich attacks, and so on.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices and guidelines and code quality standards. We also provide suggestions for possible optimizations, such as gas optimization, upgradability weaknesses, centralization risks, and so on.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no

hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood. We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

OFTWrapperUpgradeable Contracts

Type	Solidity
Platform	EVM-compatible
Target	Only changes between 64d8dc5...2c919aa
Repository	https://github.com/paxosglobal/cross-chain-contracts-internal/ ↗
Version	`2c919aaeb4f14f7ff07aa60acdf6daec2529bf47`
Programs	`OFTWrapperUpgradeable.sol`

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.5 person-days. The assessment was conducted by two consultants over the course of one calendar day.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
Engagement Manager
jacob@zellic.io

The following consultants were engaged to conduct the assessment:

Filipe Alves
Engineer
filipe@zellic.io

Chongyu Lv
Engineer
chongyu@zellic.io

2.5. Project Timeline

The key dates of the engagement are detailed below.

February 27, 2026	Start of primary review period
February 27, 2026	End of primary review period

3. Detailed Findings

3.1. Test suite lacks integration tests against USDG token

Target	Project-wide		
Category	Code Maturity	**Severity**	Low
Likelihood	Low	**Impact**	Low

Description

The test suite uses a simplified PaxosTokenFixture that overrides `increaseSupplyToAddress()` and `decreaseSupplyFromAddress()` with minimal logic, bypassing supply-control checks, frozen address validation, and balance tracking. The OFT wrapper is intended to support both PYUSD (PaxosTokenV2) and USDG (PaxosTokenClaimableRewards). The USDG implementation of these functions contains additional logic not present in PaxosTokenV2, including payout group tracking and reward accounting via `_updateWalletWithPayoutGroup()`. None of this behavior is exercised by the test fixture, which means the test suite does not verify that the OFT wrapper interacts correctly with USDG's reward system during cross-chain mint and burn operations.

Impact

The simplified fixture may mask integration-specific edge cases, particularly around how USDG's reward accounting responds to the mint and burn patterns produced by cross-chain transfers.

Recommendations

Consider adding integration tests that deploy the OFT wrapper against the real PaxosTokenClaimableRewards contract. Here are some examples of scenarios worth covering:

- **Mint/burn with active payout group.** Verify that `_credit()` and `_debit()` correctly update payout group shares and balances when the recipient/sender is assigned to a payout group with a nontrivial multiplier.
- **Mint/burn with orphaned payout group.** Verify that cross-chain transfers to a wallet whose payout group has been deleted trigger the cleanup path in `_updateWalletWithPayoutGroup()` without reverting.
- **Multiplier changes between operations.** Verify that share recalculation and reward preservation logic (via `SharesLib.updateSharesWithRewardPreservation()`) behave correctly when a payout group multiplier changes between successive cross-chain transfers.
- **Frozen address handling.** Verify that `_credit()` reverts when the destination address is frozen on the receiving chain and that `_debit()` reverts when the sender is frozen.

- **Full cross-chain round-trip with USDG.** Deploy wrappers on both chains with USDG, assign addresses to payout groups, and verify that hierarchical payout group balances and shares remain consistent after a send-and-receive cycle.

Remediation

This issue has been acknowledged by Paxos, and a fix was implemented in commit 8c295018 ↗.

3.2. No admin function to reset rate-limit capacity

Target	OFTWrapperUpgradeable		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

OFTWrapperUpgradeable exposes `setOutboundRateLimits()` and `setInboundRateLimits()` for the owner to configure rate-limit parameters, but it does not expose any way to reset the `amountInFlight` counter to zero. LayerZero's RateLimiter base contract provides an internal `_resetRateLimits()` function for this purpose, but OFTWrapperUpgradeable does not wrap it with an external entry point.

When the owner lowers a rate limit below the existing `amountInFlight`, all transfers to that endpoint are blocked until natural decay reduces the in-flight amount below the new limit.

For example, suppose `limit = 200` and `amountInFlight = 150`, and the admin sets `limit = 100`. As a result, `amountInFlight` (150) becomes greater than `limit` (100), and all transfers are blocked. To recover, there is a waiting period for natural decay to bring `amountInFlight` below the new limit; the wait time depends on the configured window duration.

Impact

Without an exposed reset function, the admin has no escape hatch to immediately restore transfer capacity after lowering rate limits. This forces reliance on natural decay, which may result in extended periods of blocked transfers during emergency response or operational adjustments.

Recommendations

Consider exposing `_resetRateLimits()` through owner-restricted external functions for both outbound and inbound rate limits, allowing administrators to immediately restore transfer capacity in emergency or operational scenarios without waiting for natural decay.

Remediation

The Paxos team acknowledged the finding and accepted it with no code changes planned. The team considers the natural decay behavior to be the intended security posture: when a limit is lowered, blocking transfers until decay catches up prevents exceeding the new exposure target.

The team identified one edge case — an emergency pause (`limit=0`) followed by a resume —

where decay freezes because `decay = (limit * time) / window = 0`, causing `amountInFlight` to never decrease. However, they noted two workarounds: if the new limit exceeds the frozen `amountInFlight`, transfers resume immediately; otherwise, the admin can temporarily set a higher limit to force decay, then set the desired limit.

The team opted not to expose `_resetRateLimits()`. They stated this is because it would allow an admin to zero out `amountInFlight` immediately after a large outflow, effectively defeating the rate limit.

3.3. Token-type declaration does not reflect multitoken deployment

Target	OFTWrapperUpgradeable		
Category	Code Maturity	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The OFT wrapper declares the underlying token reference as `PaxosTokenV2`, but it is intended to support multiple Paxos token types. PYUSD uses PaxosTokenV2 while USDG uses PaxosTokenClaimableRewards, and PaxosTokenClaimableRewards does not inherit from PaxosTokenV2. The bridging functionality is unaffected since both contracts share the same `increaseSupplyToAddress()` and `decreaseSupplyFromAddress()` function signatures, but the declared type does not accurately describe the actual contract in USDG deployments.

Impact

There is no functional or security impact, but this issue may affect code readability and maintainability.

Recommendations

Consider introducing a shared interface (for example, `IPaxosToken`) that both token implementations satisfy and using it as the declared type for the `paxosToken` state variable.

Remediation

This issue has been acknowledged by Paxos.

3.4. Uninitialized rate limits block cross-chain transfers until explicitly configured

Target	OFTWrapperUpgradeable		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

Rate limits are not set by `OFTWrapperUpgradeable.initialize()`. In LayerZero's `RateLimiter`, uninitialized limits default to `limit = 0`, which causes `_outflow()` to revert with `RateLimitExceeded` for any nonzero transfer amount. This means there is a window between deployment and the admin calling `setOutboundRateLimits()` / `setInboundRateLimits()` where all bridge operations are blocked — for outbound, this simply reverts with no fund loss, but for inbound, tokens already burned on the source chain cannot be minted on the destination until limits are configured. This issue only exists within this specific window; once rate limits are set, everything operates normally.

Impact

This introduces a deployment sequencing requirement: operators must configure inbound and outbound rate limits before treating the wrapper as live on any chain. If they do not, outbound transfers simply revert, while inbound deliveries can remain temporarily unavailable until the destination-side limits are configured and the message is retried.

Recommendations

Consider including rate-limit configuration in `initialize()` so that both inbound and outbound limits are set atomically with the rest of the proxy setup.

Remediation

This issue has been acknowledged by Paxos.

3.5. Unchecked return values from Paxos token mint and burn calls

Target	OFTWrapperUpgradeable		
Category	Coding Mistakes	**Severity**	Informational
Likelihood	N/A	**Impact**	Informational

Description

The `_debit()` and `_credit()` functions call `decreaseSupplyFromAddress()` and `increaseSupplyToAddress()` on the underlying Paxos token without checking the `bool` return value.

Impact

The current token implementations (as of the time of writing) revert on all failure paths, so ignoring the return value has no practical impact. The unchecked return value is currently effectively harmless.

However, there is risk if future token implementations deviate from this behavior. Since the wrapper is designed to support multiple token types, future token implementations that return `false` instead of reverting would silently succeed, potentially leading to accounting inconsistencies between the bridge and the underlying token supply.

Recommendations

Consider checking the return value of `decreaseSupplyFromAddress()` and `increaseSupplyToAddress()` and reverting on `false` as a defensive measure against future token implementation changes.

Remediation

This issue has been acknowledged by Paxos.

4. Threat Model

This provides a full threat model description for various functions. As time permitted, we analyzed each function in the contracts and created a written threat model for some critical functions. A threat model documents a given function's externally controllable inputs and how an attacker could leverage each input to cause harm.

Not all functions in the audit scope may have been modeled. The absence of a threat model in this section does not necessarily suggest that a function is safe.

4.1. Module: OFTWrapperUpgradeable

Function: `_credit(address _to, uint256 _amountLD, uint32 _srcEid)`

This internal function is the inbound (receive-side) core logic override for OFTCore. It is invoked by `OFTCore._lzReceive()` when a LayerZero message arrives from a remote chain, instructing the local contract to mint tokens for the recipient. The execution follows two steps in order:

1. First, `_outflow(_srcEid + INBOUND_EID_OFFSET, _amountLD)` checks and updates the inbound rate limiter using the offset key, reverting with `RateLimitExceeded` if the amount exceeds available inbound capacity.
2. Then, `paxosToken.increaseSupplyToAddress(_amountLD, _to)` mints `_amountLD` tokens to the recipient on the underlying Paxos token contract. The function returns `_amountLD` as the credited amount without any further transformation.

Inputs

- `_to`
  - **Control**: Determined by the cross-chain–message payload (set by the sender on the source chain). Not directly controllable by the local chain caller.
  - **Constraints**: No explicit zero-address check in this function. The underlying `increaseSupplyToAddress` validates the recipient via a frozen-address check (`require(!_isAddrFrozen(mintToAddress))`) and SupplyControl's `canMintToAddress()` permission check.
  - **Impact**: The address that receives the newly minted tokens on the destination chain. This is the intended recipient of the cross-chain transfer.
- `_amountLD`
  - **Control**: Determined by the cross-chain–message payload (the amount that was debited/burned on the source chain, after decimal conversion).
  - **Constraints**: Must not exceed the available inbound rate-limit capacity for `_srcEid + INBOUND_EID_OFFSET`. There is no upper bound check beyond the rate limiter.
  - **Impact**: The amount of tokens to mint on the destination chain. This is added directly to `_to`'s balance via the underlying token's `increaseSupplyToAddress`.

- `_srcEid`
    - **Control**: Determined by the LayerZero endpoint based on the message's origin chain. Not controllable by the recipient.
    - **Constraints**: Must have a configured inbound rate limit (via `setInboundRateLimits`) with `limit > 0` at the offset key `_srcEid + INBOUND_EID_OFFSET`; otherwise, `_outflow` reverts with `RateLimitExceeded`.
    - **Impact**: The LayerZero endpoint ID of the source chain. Used to look up the inbound rate limit via the offset key in the `rateLimits` mapping.

Branches and code coverage

Intended branches

- ☑ Successful credit mints `_amountLD` tokens to `_to` on the underlying token.
- ☑ Inbound rate limit can be updated to allow higher amounts.
- ☐ The `amountInFlight` at the inbound offset key is incremented by `_amountLD` after a successful credit.
- ☐ Inbound rate-limit capacity decays linearly over the configured window, restoring capacity over time.
- ☐ The function returns `_amountLD` as the credited amount.

Negative behavior

- ☑ Reverts with `RateLimitExceeded` when `_amountLD` exceeds available inbound capacity.
- ☑ Reverts when source `_srcEid` has no configured inbound rate limit (default `limit=0`, blocked by default).
- ☐ Behavior when `_to` is `address(0)` (depends on underlying token implementation — no check in this contract).
- ☐ If the inbound rate limit is exhausted, the LayerZero message fails to deliver and the user's tokens remain burned on the source chain until limits are reconfigured and the message is retried.

Function: `_debit(address _from, uint256 _amountLD, uint256 _minAmountLD, uint32 _dstEid)`

This internal function is the outbound (send-side) core logic override for OFTCore. It is invoked by `OFTCore.send()` when a user initiates a cross-chain token transfer. The execution follows three steps in order:

1. First, `_debitView(_amountLD, _minAmountLD, _dstEid)` computes the actual send amount and received amount after applying any decimal conversion or dust removal.
2. Next, `_outflow(_dstEid, amountSentLD)` checks and updates the outbound rate limiter for the destination endpoint, reverting with `RateLimitExceeded` if the amount

exceeds available capacity.

3. Lastly, `paxosToken.decreaseSupplyFromAddress(amountSentLD, _from)` burns the tokens from the sender's balance on the underlying Paxos token contract. The burn is irreversible — once this function returns, the tokens are destroyed on the source chain and a LayerZero message is dispatched to mint equivalent tokens on the destination.

Inputs

- `_from`
  - **Control**: Determined by `OFTCore.send()` based on `msg.sender`. Not directly controllable by external callers beyond the send entry point.
  - **Constraints**: Must hold at least `amountSentLD` balance on `paxosToken`. The underlying `decreaseSupplyFromAddress` reverts if the balance is insufficient.
  - **Impact**: The address whose tokens are burned (destroyed) on the source chain. This is the sender of the cross-chain transfer.
- `_amountLD`
  - **Control**: Full control (passed from the user's `send()` call).
  - **Constraints**: Must be greater than or equal to `_minAmountLD` after `_debitView` processing. Must not exceed the available outbound rate-limit capacity for `_dstEid`. Must not exceed `_from`'s balance on `paxosToken`.
  - **Impact**: The requested amount to bridge out in local decimals. After `_debitView`, the actual sent amount may differ due to decimal-conversion rounding (dust removal).
- `_minAmountLD`
  - **Control**: Full control (passed from the user's `send()` call).
  - **Constraints**: `_debitView` reverts if the computed `amountReceivedLD < _minAmountLD` (slippage protection).
  - **Impact**: The minimum amount the user is willing to accept on the destination chain. Acts as slippage protection against unfavorable decimal conversion or dust removal.
- `_dstEid`
  - **Control**: Full control (passed from the user's `send()` call).
  - **Constraints**: Must have a configured outbound rate limit with `limit > 0`; otherwise, `_outflow` reverts with `RateLimitExceeded` (default `limit=0` blocks all transfers).
  - **Impact**: The LayerZero endpoint ID of the destination chain. Used as the key to look up the outbound rate limit in the `rateLimits` mapping.

Branches and code coverage

Intended branches

- ☑ Successful debit reduces `_from`'s balance by `amountSentLD` on the underlying token.
- ☑ Rate limit is enforced; debit succeeds when the amount is within the configured limit.
- ☑ After raising the outbound rate limit, previously blocked amounts can be debited.
- ☐ Both `amountSentLD` and `amountReceivedLD` reflect any decimal-conversion rounding correctly.
- ☐ The `amountInFlight` in the rate limiter is incremented by `amountSentLD` after a successful debit.
- ☐ Outbound rate-limit capacity decays linearly over the configured window, restoring capacity over time.

Negative behavior

- ☑ Reverts with `RateLimitExceeded` when `amountSentLD` exceeds available outbound capacity.
- ☐ Reverts when `_from` has insufficient balance on `paxosToken` (underlying `decreaseSupplyFromAddress` fails).
- ☐ Reverts when `amountReceivedLD < _minAmountLD` (slippage protection in `_debitView`).
- ☐ Reverts when destination `_dstEid` has no configured rate limit (default `limit=0`).

Function: `setInboundRateLimits(RateLimitConfig[] calldata _rateLimitConfigs)`

This function allows the contract owner to configure inbound (receive-side) rate limits for one or more source endpoint IDs. Internally, it delegates to `_setInboundRateLimits()`, which adds `INBOUND_EID_OFFSET` (1,000,000,000) to each config's `dstEid` field before writing to the shared `rateLimits` mapping via `RateLimiter._setRateLimits()`. This offset-based approach is intended to prevent inbound and outbound rate-limit keys from colliding in the same `uint32 -> RateLimit` mapping. After writing, the function emits `InboundRateLimitsChanged` with the *original* (un-offset) configs. Like `setOutboundRateLimits`, this does not reset `amountInFlight`.

Inputs

- `_rateLimitConfigs`
  - **Control**: Full control (owner only).
  - **Constraints**: Protected by the `onlyOwner` modifier. Each element's `dstEid` field is used as the source endpoint ID (`srcEid`). The internal offset addition (`dstEid + INBOUND_EID_OFFSET`) is performed as `uint32` arithmetic. In Solidity 0.8+, this addition is checked by default — if `dstEid + 1_000_000_000 > type(uint32).max` (i.e., `dstEid > 3,294,967,295`), the call reverts with an arithmetic overflow rather than wrapping. In practice,

LayerZero endpoint IDs are sequentially assigned starting from low values, so this is unreachable.

- **Impact**: Determines the maximum amount of tokens that can be received (minted) from each source chain within the specified time window. Controls inbound flow protection. Setting `limit=0` effectively blocks all inbound minting from that source.

Branches and code coverage

Intended branches

- ☑ The inbound rate limit is correctly stored and readable via `inboundRateLimits(srcEid)`.
- ☑ The rate limit can be updated to a higher value, allowing previously blocked inbound credits to succeed.
- ☐ The `InboundRateLimitsChanged` event is emitted with the original (un-offset) config values.
- ☐ Multiple configs can be set in a single call.
- ☐ The existing `amountInFlight` is preserved when updating.

Negative behavior

- ☑ A non-owner caller is rejected (reverts).
- ☐ The `dstEid + INBOUND_EID_OFFSET` overflow reverts due to Solidity 0.8+ checked arithmetic (no silent wrapping).
- ☐ Setting an inbound limit for `srcEid=X` overwrites a previously set outbound limit for `dstEid = X + INBOUND_EID_OFFSET` if both target the same storage slot.

Function: `setOutboundRateLimits(RateLimitConfig[] calldata _rateLimitConfigs)`

This function allows the contract owner to configure outbound (send-side) rate limits for one or more destination endpoint IDs. Each `RateLimitConfig` specifies a `dstEid`, a `limit` (maximum amount allowed in the window), and a `window` (duration in seconds over which the limit applies with linear decay). Internally, it delegates to `RateLimiter._setRateLimits()`, which updates the `limit` and `window` fields in the `rateLimits` mapping but explicitly does *not* reset `amountInFlight` or `lastUpdated`. This means lowering a limit below the current `amountInFlight` will block all outbound transfers until natural decay catches up, with no admin escape hatch to force a reset (see Finding 3.5. ↗).

Inputs

- `_rateLimitConfigs`
  - **Control**: Full control (owner only).

- **Constraints**: Protected by the `onlyOwner` modifier. Each element's `dstEid` is a `uint32` used as the key in the `rateLimits` mapping. The `limit` is `uint192` and `window` is `uint64`. There is no validation that `dstEid` falls below `INBOUND_EID_OFFSET` (1,000,000,000), so a sufficiently high `dstEid` could theoretically collide with an inbound rate-limit key at `srcEid + INBOUND_EID_OFFSET`. In practice, LayerZero endpoint IDs are sequentially assigned starting from low values, making this unreachable.
- **Impact**: Determines the maximum amount of tokens that can be bridged out to each destination chain within the specified time window. Setting `limit=0` effectively blocks all outbound transfers to that destination. Setting the limit too high removes rate-limiting protection for that route.

Branches and code coverage

Intended branches

- ☑ The rate limit is correctly stored for the specified `dstEid` (limit and window values match after call).
- ☑ The rate limit can be updated to a higher value, allowing previously blocked transfers to succeed.
- ☐ Multiple rate-limit configs can be set in a single call (batch configuration).
- ☐ The existing `amountInFlight` is preserved when updating rate limits (not reset).
- ☐ The `RateLimitsChanged` event is emitted with the correct config values.
- ☐ Empty array input does not revert (no-op).

Negative behavior

- ☑ A non-owner caller is rejected (reverts).
- ☐ Setting `dstEid >= INBOUND_EID_OFFSET` does not revert but may collide with inbound rate-limit keys.
- ☐ Lowering `limit` below the current `amountInFlight` does not revert but blocks all subsequent outbound transfers until decay.

5. Assessment Results

During our assessment on the scoped OFTWrapperUpgradeable contracts, we discovered five findings. No critical issues were found. One finding was of low impact and the other findings were informational in nature.

The codebase is compact and well-structured. The changes introduced in this diff, namely the UUPS upgradability pattern and the inbound rate-limiting mechanism, follow established patterns from OpenZeppelin and LayerZero, respectively.

We recommend expanding test coverage to include integration tests against USDG (PaxosTokenClaimableRewards). The existing test suite uses a simplified PaxosTokenFixture, but USDG has different logic in `increaseSupplyToAddress()` and `decreaseSupplyFromAddress()` compared to PaxosTokenV2. Since the OFT wrapper is intended to support both token types, integration tests that exercise the full send and receive flow against each real token implementation would increase confidence that the bridging behavior is correct across all deployments.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.